Laboratorio 3
Tecnologías de Microprocesamiento

Sergio Romero, Anthony Bonino,Santiago Moizo
Ingeniería Mecatrónica – UTEC
Semestre IV
Docentes: Jesús Martínez / Marcelo Díaz

***Resumen*—En este laboratorio se desarrollan cuatro aplicaciones utilizando el microcontrolador ATmega328P programado en lenguaje C. Las actividades integran conversión analógico-digital, PWM, comunicación UART, sensores, motores, servomotores, pantalla LCD y otros periféricos. El Problema A implementa un control automático de temperatura; el Problema B, un control de iluminación con setpoint remoto y persiana motorizada; el Problema C, un clasificador de color de plásticos; y el Problema D, un piano electrónico con selección de canciones por UART. Los desarrollos se verifican mediante simulación, montaje físico, comunicación serial y evidencias experimentales.**

***Palabras clave*—ATmega328P, lenguaje C, ADC, PWM, UART, LDR, L298N, DHT11, WS2812, servomotor, LCD, buzzer.**

I. Introducción

El Laboratorio 3 integra diferentes recursos del ATmega328P para resolver problemas de adquisición, control y comunicación. En cada actividad se identifican entradas y salidas, se implementa el programa en lenguaje C, se verifica el comportamiento mediante simulación o montaje físico y se registra la evolución del código mediante GitHub.

II. Objetivos

II-A. Objetivo general

Diseñar e implementar aplicaciones embebidas con el ATmega328P utilizando sensores, actuadores, ADC, PWM, UART y periféricos de visualización.

II-B. Objetivos específicos

- Programar el ATmega328P en lenguaje C.
- Utilizar ADC para adquirir variables analógicas.
- Generar señales PWM para controlar actuadores.
- Implementar comunicación UART.
- Diseñar máquinas de estados para organizar el funcionamiento.
- Verificar los sistemas mediante simulación, montaje y pruebas.
- Registrar código y modificaciones significativas mediante GitHub.

III. Materiales

- Arduino UNO con ATmega328P.
- Protoboard, cables y fuente de alimentación.
- Sensor de temperatura DHT11/DHT22 o LM35.
- LCD 16x2, con módulo adaptador I2C HW-61 (expansor PCF8574) en los Problemas A y C.
- LDR y resistencia de $10\,k\Omega$.
- Potenciómetro de $10\,k\Omega$.
- Motor DC y módulo puente H L298N.
- Fotocelda y tira de LEDs WS2812.
- Servomotor SG90.
- LEDs de estado y resistencias limitadoras.
- Pulsadores y buzzers pasivos.
- Transistores 2N2222 y resistencias de $1\,k\Omega$.
- Computadora con Microchip Studio y MATLAB o Python.

IV. Metodología general

Para cada problema se analiza la consigna, se identifican entradas y salidas, se define la lógica de funcionamiento, se asignan pines, se implementa el programa en C, se compila en Microchip Studio y se verifica mediante simulación o montaje físico. Finalmente se documentan los resultados, evidencias y modificaciones realizadas en GitHub.

V. Problema A: Sistema de control de temperatura

V-A. Descripción y funcionamiento

El sistema utiliza el microcontrolador ATmega328P para medir periódicamente la temperatura ambiente mediante un sensor digital DHT11 y controlar automáticamente un calefactor y un ventilador, con el objetivo de mantener la temperatura dentro de un rango determinado. El calefactor funciona mediante control de encendido y apagado, mientras que el ventilador dispone de tres velocidades, asociadas a niveles de PWM de referencia de 102, 178 y 255. En el montaje, el calefactor y las tres velocidades se emulan mediante LEDs de estado.

Para un punto medio de $20\,^{\circ}$C, valor utilizado en las pruebas, el comportamiento establecido es el siguiente:

Cuadro I
ACCIONES DEL SISTEMA SEGÚN LA TEMPERATURA

Temperatura	Acción
0–15 °C	Calefactor encendido
16–25 °C	Calefactor y ventilador apagados
26–35 °C	Ventilador a baja velocidad
36–45 °C	Ventilador a velocidad media
Mayor a 45 °C	Ventilador a alta velocidad

El usuario puede modificar el punto medio mediante UART, y los límites se recalculan a partir del nuevo valor. La pantalla LCD 16x2 muestra la temperatura actual y el rango de confort ($PM-5$ a $PM+5$), mientras que por UART se informa la temperatura medida y la acción aplicada sobre los actuadores.

Las principales conexiones utilizadas se presentan en la Tabla II.

Cuadro II
ASIGNACIÓN DE PINES DEL PROBLEMA A

Elemento	Arduino UNO	ATmega328P
Sensor DHT11 (DATA)	A0	PC0
LED calefactor	D9	PB1
LED ventilador baja	D10	PB2
LED ventilador media	D11	PB3
LED ventilador alta	D12	PB4
LCD I2C (SDA)	A4	PC4 / SDA
LCD I2C (SCL)	A5	PC5 / SCL
UART RX	D0	PD0 / RXD
UART TX	D1	PD1 / TXD

V-B. Estrategia de control

La temperatura se adquiere cada 5 s y se compara directamente con los umbrales definidos a partir del punto medio PM, separados entre sí por 10 °C y con la zona de reposo centrada en PM (± 5 °C):

$$\begin{aligned} U_{\text{Calefactor}} &= PM - 5 \\ U_{\text{Fan_baja}} &= PM + 5 \\ U_{\text{Fan_media}} &= PM + 15 \\ U_{\text{Fan_alta}} &= PM + 25 \end{aligned} \tag{1}$$

donde $U_{\text{Calefactor}}$ es el umbral por debajo del cual (inclusive) se enciende el calefactor y $U_{\text{Fan_baja}}$, $U_{\text{Fan_media}}$ y $U_{\text{Fan_alta}}$ son los umbrales que, al ser superados, hacen que el ventilador pase a cada velocidad. La Tabla III resume la condición, las salidas y el PWM de referencia de cada estado.

Cuadro III
ESTADOS, CONDICIONES Y SALIDAS DEL PROBLEMA A

Estado	Condición	Salidas	PWM
Calefactor	$T \leq PM - 5$	Calefactor	0
Reposo	$PM - 5 < T \leq PM + 5$	Ninguna	0
Fan_baja	$PM + 5 < T \leq PM + 15$	1 nivel	102
Fan_media	$PM + 15 < T \leq PM + 25$	2 niveles	178
Fan_alta	$T > PM + 25$	3 niveles	255
Error	Fallo de lectura	Ninguna	0

No se incorporó histéresis: el DHT11 entrega valores enteros de grado y el actuador trabaja con niveles discretos, por lo que la comparación directa de umbrales resulta suficiente.

El rango admisible del punto medio se deriva, en su límite superior, de los límites físicos del sistema. Dado que el máximo del DHT11 es 50 °C y el umbral superior es $PM+25$, se define:

$$PM_{MAX} = 50 - 25 = 25\,^{\circ}\text{C} \qquad PM_{MIN} = 10\,^{\circ}\text{C} \tag{2}$$

De esta forma el umbral de Fan_alta nunca supera el máximo medible por el sensor y, si cambian el sensor o el ancho de las bandas, el límite superior se recalcula automáticamente al recompilar. El límite inferior se fijó en 10 °C como valor de diseño. Un valor fuera de este rango, por ejemplo $PM = 40$ (que implicaría un umbral de 65 °C inalcanzable), se rechaza con un mensaje de error y se conserva el punto medio anterior. En el caso límite $PM = 25$, el umbral de Fan_alta coincide con el máximo medible, por lo que ese estado solo sería alcanzable en el borde del rango del sensor.

V-C. Máquina de estados

La lógica del sistema se modela mediante una máquina de estados de seis estados: Calefactor, Reposo, Fan_baja, Fan_media, Fan_alta y Error.

Al energizarse, el sistema parte del estado Reposo. A partir del primer muestreo, la temperatura T determina el estado de operación: en cada muestreo se evalúan los umbrales y, si T cruza alguno de ellos, se produce la transición correspondiente. Ante variaciones abruptas de temperatura, la transición puede saltar uno o varios estados intermedios. Si T permanece dentro del rango del estado actual, el sistema no cambia de estado.

El estado Error es alcanzable desde cualquier otro estado cuando falla la lectura del sensor, ya sea por tiempo de espera agotado (*timeout*), por *checksum* inválido o por una trama de solo ceros. Al entrar en este estado se desactivan todas las salidas, siguiendo un criterio de seguridad ante fallas (*fail-safe*), y se informa un código de error por UART y por la LCD. El sistema abandona el estado Error con la primera lectura válida y pasa directamente al estado que indique dicha medición.

El diagrama completo, que detalla las condiciones de cada una de las transiciones entre los seis estados, se encuentra disponible en el siguiente enlace.

Acceder a la imagen y al archivo Draw.io en GitHub

V-D. Implementación y comunicación

El sistema fue implementado en lenguaje C para el microcontrolador ATmega328P. El calefactor se controla mediante una salida digital y las tres velocidades del ventilador se señalizan mediante LEDs de estado acumulativos. Los niveles de PWM (102, 178 y 255) se definen por software como referencia de cada velocidad, pero no se generan por hardware en esta versión.

El protocolo del DHT11 impone ventanas de tiempo de microsegundos, por lo que la lectura de la trama de 40 bits se realiza con las interrupciones deshabilitadas (aproximadamente 25 ms) y con tiempos de espera de 150 μs en cada transición de nivel para no bloquear el sistema ante un sensor desconectado. Se leen los 40 bits completos porque el checksum se calcula sobre los cinco bytes; la humedad se descarta una vez verificado, ya que el control depende solo de la temperatura. La base de tiempo de 1 s se genera con el Timer1 en modo CTC (OCR1A = 15624, prescaler 1024), y cada cinco interrupciones se solicita una medición.

La comunicación USART se configuró a 9600 baudios con formato 8N1 y se utiliza en ambos sentidos: recibe el nuevo punto medio y transmite, tras cada muestreo (cada 5 s), una línea con la temperatura medida y la acción aplicada:

```
Temp=24 C | Accion=REPOSO
```

Ante una falla del sensor se transmite en cambio un mensaje con el código de error (`MSG,ERROR DHT11 codigo=N`), que corresponde al tipo de falla: *timeout* en alguna de las etapas del protocolo, *checksum* incorrecto o trama de solo ceros.

El menú de configuración acepta el comando `P`, que habilita el cambio del punto medio y pausa el sensado hasta recibir dos dígitos válidos. La recepción se valida íntegramente en la rutina de interrupción de la UART:

- la `P` solo es válida como primer carácter de la línea, por lo que cadenas con `P` embebida se rechazan;
- el menú se habilita únicamente si se recibe exactamente una `P`, por lo que las repeticiones se rechazan;
- durante el ingreso del punto medio solo se aceptan dígitos; cualquier otro carácter cancela el modo y debe reenviarse `P`;
- todo rechazo se notifica por UART sin dejar estados residuales.

Si el valor ingresado está fuera del rango de 10 a 25 °C, se informa el error y el sistema permanece en el modo de ingreso, con el sensado pausado, hasta recibir un valor válido. Al aceptarse un nuevo punto medio, los umbrales se recalculan de inmediato y se reinicia el período de muestreo, por lo que se aplican en la siguiente medición.

La pantalla LCD 16x2 se conecta mediante un módulo adaptador I2C HW-61, basado en el expansor PCF8574. La comunicación se realiza mediante el periférico TWI a 100 kHz. El controlador compatible con HD44780 se inicializa en modo de 4 bits.

V-E. Montaje y resultados

En el montaje físico (Fig. 1), el calefactor y las tres velocidades del ventilador se emulan mediante LEDs de estado.

Las pruebas consistieron en aplicar variaciones escalón de temperatura y registrar la respuesta mediante el reporte serie. Al alcanzar $PM - 5$ o menos, el sistema conmutó a Calefactor en el siguiente muestreo, con un tiempo de reacción máximo acorde al período de 5 s. Al superar los umbrales de ventilación, los LEDs indicaron la velocidad correspondiente, la acción informada por UART cambió de FAN BAJA a FAN MEDIA y FAN ALTA, y la LCD actualizó la información mostrada.

El punto medio se modificó en línea mediante el comando `P` seguido de `22`, verificándose el recálculo de los umbrales, aplicados desde el siguiente muestreo, y la reanudación del sensado. Los valores $PM = 40$ y menores a 10 fueron rechazados con el mensaje de error correspondiente, y las cadenas corruptas, las repeticiones del comando y los caracteres inválidos fueron rechazados sin habilitar el menú ni alterar el estado del sistema. La desconexión deliberada del sensor provocó el paso al estado Error, y el control automático se recuperó al restablecer la conexión.

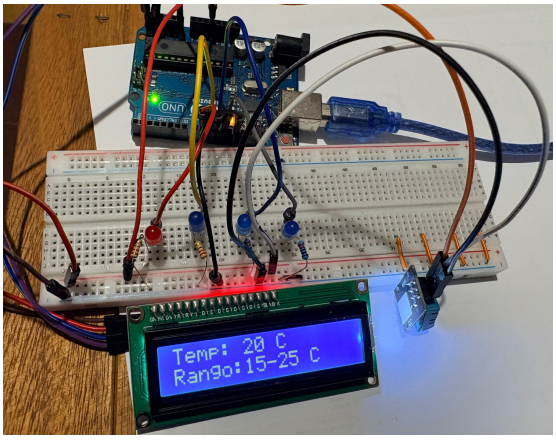


Figura 1. Montaje físico.

V-F. Monitoreo de variables

Las variables transmitidas por UART pueden observarse en tiempo real desde la computadora, lo que permite verificar la temperatura medida y la acción aplicada sobre el calefactor y el ventilador. El punto medio configurado se confirma por UART cada vez que se modifica.

Ver captura de monitoreo

V-G. Representación gráfica

El registro recibido se almacena en un archivo CSV y se representa mediante Python (matplotlib) para analizar el comportamiento del sistema a lo largo del tiempo. Se generan dos gráficas separadas: la temperatura junto con el punto medio, en una misma escala en grados Celsius, y la secuencia de acciones del calefactor y del ventilador como señal

escalonada categórica. El punto medio se mantuvo constante durante el registro y se incorpora a la gráfica como referencia. La respuesta ante escalones de temperatura se registró con $PM = 20\,°\text{C}$.

Las gráficas obtenidas durante las pruebas y el código desarrollado en Python para la adquisición y representación de las variables se encuentran disponibles en el repositorio GitHub mediante los siguientes accesos.

Las gráficas obtenidas durante las pruebas y el código desarrollado en Python para la adquisición y representación de las variables se encuentran disponibles en el repositorio GitHub mediante los siguientes accesos.

Ver captura de la gráfica

Acceder al código Python de la gráfica

V-H. *Simulación*

Antes de realizar la implementación física se verifica el funcionamiento del programa mediante simulación. Esta etapa permite comprobar las condiciones de temperatura, la activación del calefactor, la lógica de las tres velocidades del ventilador y la comunicación serial.

Acceder al archivo de Proteus y capturas de la simulación

V-I. *Limitaciones*

La precisión del sistema depende de la del DHT11, que entrega valores enteros con resolución de 1 °C. El período de muestreo de 5 s introduce una latencia máxima de reacción igual a dicho período, aceptable para la dinámica térmica de una habitación pero mejorable para procesos más rápidos; además, la respuesta final depende de la inercia térmica del ambiente.

La ausencia de histéresis puede provocar conmutaciones alternadas si la temperatura oscila sobre un umbral, aunque el riesgo es acotado por la resolución del sensor. Asimismo, el control por estados discretos presenta un error estático en régimen permanente, tolerable en control de confort.

Durante la lectura del DHT11 (unos 25 ms) las interrupciones permanecen deshabilitadas, por lo que un carácter enviado por UART justo en ese intervalo podría perderse. Por otra parte, el ventilador se emula con LEDs y los niveles de PWM son valores de referencia, por lo que la velocidad real de un ventilador no fue verificada en esta versión.

V-J. *Video de funcionamiento*

Como evidencia final se incluyen videos demostrativos del sistema completo: medición de temperatura, respuesta del calefactor y del ventilador, modificación del punto medio mediante UART, gráficos en tiempo real e información mostrada en la LCD.

Ver video de funcionamiento (parte 1)

Ver video de funcionamiento (parte 2)

V-K. *Repositorio GitHub*

El código fuente correspondiente al Problema A se encuentra disponible en el repositorio GitHub del laboratorio.

Acceder al código fuente

VI. Problema B: Control de iluminación con setpoint remoto por UART

VI-A. *Descripción y funcionamiento*

El usuario ingresa mediante UART un valor de iluminación deseado o *setpoint*. Una LDR conectada al canal ADC0 mide la iluminación real, mientras que un potenciómetro conectado al canal ADC1 permite conocer la posición del mecanismo.

Un motor de corriente continua, controlado mediante un puente H L298N, se encarga de abrir o cerrar una persiana o mecanismo equivalente. El microcontrolador compara la iluminación medida con el valor deseado en cada ciclo de control, de 500 ms, y determina si debe abrir, cerrar o detener el motor.

Las principales conexiones utilizadas se presentan en la Tabla IV.

Cuadro IV
Asignación de pines del Problema B

Elemento	Arduino UNO	ATmega328P
LDR	A0	PC0 / ADC0
Potenciómetro	A1	PC1 / ADC1
L298N IN1	D4	PD4
L298N IN2	D5	PD5
L298N ENA	D6	PD6 / OC0A
UART RX	D0	PD0 / RXD
UART TX	D1	PD1 / TXD

La LDR se conecta formando un divisor resistivo junto con una resistencia de $10\,k\Omega$. El potenciómetro actúa como sensor de posición y permite determinar los límites físicos del recorrido.

VI-B. *Estrategia de control*

El error entre la iluminación deseada y la iluminación medida se define como:

$$e = SP - LDR \tag{3}$$

donde SP representa el setpoint recibido mediante UART y LDR corresponde al valor obtenido mediante ADC0.

Para evitar movimientos continuos del motor alrededor del punto de referencia, se implementó una zona muerta de ± 40 unidades ADC. Por lo tanto, cuando:

$$|e| \leq 40 \tag{4}$$

el motor permanece detenido.

Para un setpoint de:

$$SP = 500 \tag{5}$$

el intervalo de parada corresponde aproximadamente a:

$$460 \leq LDR \leq 540 \tag{6}$$

Cuando el motor debe moverse, el valor PWM se calcula mediante:

$$PWM = \frac{|e|}{4} \tag{7}$$

El valor obtenido se limita entre 200 y 255. Durante las pruebas físicas se comprobó que valores menores a 200 no proporcionaban suficiente par para iniciar correctamente el movimiento del motor.

Dado que $|e|/4$ solo supera el valor mínimo de 200 cuando $|e| \geq 800$, en la práctica el motor opera casi siempre con PWM = 200, por lo que el control se comporta como un esquema de encendido y apagado con zona muerta, y el término proporcional solo actúa ante errores muy grandes.

Los límites de posición utilizados son:

$$POS_{MIN} = 100 \qquad POS_{MAX} = 900 \tag{8}$$

Estos límites permiten detener el motor cuando el mecanismo alcanza la máxima apertura o el máximo cierre.

VI-C. Máquina de estados

El funcionamiento del sistema se representa mediante tres estados principales:

- **S0 – PARADO:** el motor permanece detenido cuando la iluminación se encuentra dentro de la zona muerta o cuando se alcanza uno de los límites físicos del recorrido.
- **S1 – ABRIENDO:** el sistema entra en este estado cuando $e > 40$ y $POS < 900$.
- **S2 – CERRANDO:** el sistema entra en este estado cuando $e < -40$ y $POS > 100$.

El sistema puede cambiar directamente entre apertura y cierre si cambia el signo del error. El sentido de giro depende del estado actual y el valor PWM se calcula a partir del valor absoluto del error.

El diagrama completo representa las condiciones utilizadas para realizar las transiciones entre los tres estados y se encuentra disponible en el siguiente acceso.

Acceder a la imagen y al archivo Draw.io en GitHub

VI-D. Implementación y comunicación

El sistema fue implementado en lenguaje C para el microcontrolador ATmega328P. Para adquirir las variables analógicas se utilizó el conversor ADC, configurado con AVcc como tensión de referencia y un prescaler de 128.

Los canales utilizados fueron:

- **ADC0:** lectura de la LDR para medir el nivel de iluminación.
- **ADC1:** lectura del potenciómetro para conocer la posición del mecanismo.

La comunicación USART se configuró a 9600 baudios, utilizando 8 bits de datos, sin paridad y un bit de parada. Para una frecuencia de reloj de 16 MHz se configuró el registro UBRR0 con el valor 103. Mediante esta comunicación el usuario puede ingresar valores de setpoint comprendidos entre 0 y 1023. El valor se envía como texto de hasta cuatro dígitos terminado con Enter (Backspace permite corregir) y se recibe por interrupción. Un valor mayor que 1023 se rechaza con un mensaje de error y se conserva el setpoint anterior. Al encender, el setpoint inicial es 500.

Para el control de velocidad del motor se utilizó Timer0 en modo Fast PWM con prescaler 64, lo que da una frecuencia de PWM de aproximadamente 976 Hz. La señal PWM se genera mediante la salida OC0A correspondiente al pin PD6 (D6 del Arduino UNO) y se aplica a la entrada ENA del puente H L298N.

El sentido de giro del motor se controla mediante las entradas IN1 e IN2 del L298N. Para la apertura se establece IN1 en nivel alto e IN2 en nivel bajo, mientras que para el cierre se invierte esta condición. Cuando el sistema se encuentra detenido, ambas señales permanecen en nivel bajo y el PWM se lleva a cero.

La lógica principal evalúa el error entre el setpoint recibido y la lectura de la LDR en cada ciclo de 500 ms. Según el signo y la magnitud del error, junto con la posición medida por el potenciómetro, el sistema selecciona los estados `PARADO`, `ABRIENDO` o `CERRANDO`.

Durante el funcionamiento se transmiten mediante UART, en cada ciclo, las principales variables del sistema:

```
SP | LDR | Pos | PWM | Estado
```

Esto permite observar en tiempo real el valor de referencia, la iluminación medida, la posición del mecanismo, el PWM aplicado y el estado actual del motor.

VI-E. Montaje y resultados

Para comprobar el funcionamiento del programa se realizó el montaje físico mostrado en la Fig. 2. En él se integraron la placa Arduino UNO, la LDR, el potenciómetro de posición, el módulo puente H L298N y el motor de corriente continua.

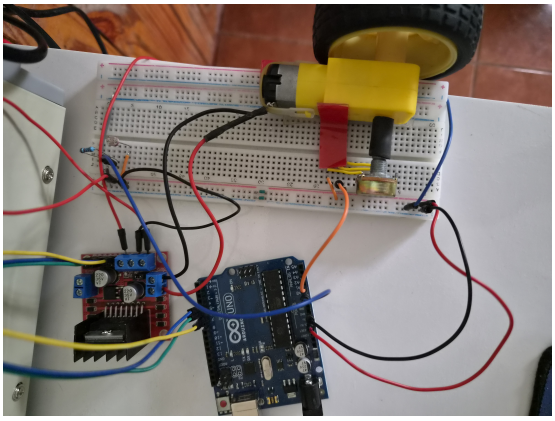


Figura 2. Montaje físico.

Durante las pruebas se verificó correctamente la lectura de la LDR, la lectura del potenciómetro, la recepción del setpoint

mediante UART y el control de los dos sentidos de giro del motor mediante el módulo L298N.

También se comprobó que el motor puede detenerse en una posición intermedia cuando la iluminación alcanza la zona definida alrededor del setpoint, sin necesidad de llegar siempre a uno de los extremos del recorrido.

El potenciómetro permite además detener el motor cuando se alcanza la máxima apertura o el máximo cierre, evitando movimientos fuera del rango permitido.

VI-F. Monitoreo de variables

Para verificar el comportamiento del sistema durante las pruebas físicas se utilizó Data Visualizer de Microchip Studio. Mediante la comunicación UART se observaron en tiempo real el setpoint, la lectura de la LDR, la posición del mecanismo, el valor PWM aplicado y el estado actual del motor.

El siguiente enlace permite ver una de las capturas obtenidas durante estas pruebas. Esta información permitió comprobar que las decisiones tomadas por el programa coincidían con las condiciones de iluminación y posición existentes en cada instante.

Ver captura de monitoreo

En particular, los mensajes transmitidos permiten identificar situaciones de apertura (`ABRIENDO`), cierre (`CERRANDO`), detención dentro de la zona del setpoint (`PARADO - SETPOINT`) y detención al alcanzar los límites físicos del mecanismo (`PARADO - MAX APERTURA` y `PARADO - MAX CIERRE`).

VI-G. Representación gráfica

Además del monitoreo mediante Data Visualizer, las variables transmitidas por UART fueron adquiridas mediante MATLAB y representadas simultáneamente para analizar la respuesta dinámica del sistema.

En la gráfica se representan el setpoint, la iluminación medida por la LDR, la posición del mecanismo y el PWM aplicado al motor. De esta manera es posible observar la relación entre las variaciones de iluminación y la acción realizada por el controlador, así como los períodos en los que el motor permanece en movimiento o detenido.

La gráfica obtenida durante las pruebas y el código desarrollado en MATLAB para la adquisición y representación de las variables se encuentran disponibles en el repositorio GitHub mediante los siguientes accesos.

Ver captura de la gráfica

Acceder al código MATLAB de la gráfica

VI-H. Simulación en Proteus

También se implementó el circuito en Proteus para verificar la lectura de las entradas analógicas, la comunicación UART, la lógica de control, la generación de PWM y las señales aplicadas al puente H L298N.

En la simulación, la LDR permite representar las variaciones de iluminación, mientras que el potenciómetro conectado a ADC1 representa la posición del mecanismo. Este último se modifica manualmente para reproducir el acoplamiento mecánico existente entre el motor y el potenciómetro en el montaje físico.

De esta forma se puede modificar tanto la iluminación como la posición del mecanismo y comprobar la respuesta del sistema ante diferentes condiciones de funcionamiento. Según el error entre el setpoint y la lectura de la LDR, el controlador puede permanecer en estado `PARADO`, `ABRIENDO` o `CERRANDO`, respetando además los límites de posición establecidos.

Acceder al archivo de Proteus y capturas de la simulación

VI-I. Limitaciones

La precisión del sistema depende de la resolución del ADC, de la respuesta de la LDR y de la medición de posición mediante el potenciómetro.

La LDR presenta una respuesta no lineal y puede verse afectada por cambios de iluminación externos, lo que puede modificar temporalmente la respuesta del controlador.

La velocidad de regulación también se encuentra limitada por las características mecánicas del motor y por el rango de PWM utilizado. Como el PWM se satura en su valor mínimo de 200 para casi todo el rango de errores, el control se aproxima a un esquema de encendido y apagado con zona muerta, y no a una regulación proporcional de la velocidad.

La recepción del setpoint se realiza por interrupción, pero el lazo de control se ejecuta cada 500 ms, por lo que un nuevo setpoint se aplica en el siguiente ciclo y la respuesta ante cambios bruscos de iluminación puede demorar hasta ese período. Además, el cambio de sentido entre apertura y cierre se produce sin tiempo muerto, lo que implica una exigencia mecánica adicional para el motor y el puente H.

VI-J. Video de funcionamiento

Como evidencia final se incluye un video demostrativo del montaje físico. En él se observa el ingreso del setpoint mediante UART, las variaciones de la lectura de la LDR, el movimiento del motor, el cambio de posición y la detención del mecanismo al alcanzar la zona del setpoint o los límites del recorrido.

Ver video de funcionamiento

VI-K. Repositorio GitHub

El código fuente correspondiente al Problema B se encuentra disponible en el repositorio GitHub del laboratorio.

Acceder al código fuente

VII. Problema C: Sistema clasificador de color de plásticos

VII-A. Descripción y funcionamiento

El sistema utiliza el microcontrolador ATmega328P a 16 MHz para identificar el color de una muestra de plástico mediante la medición de una fotorresistencia (LDR) iluminada

sucesivamente con cuatro colores de luz. La muestra se clasifica entre cuatro tipos previamente calibrados (naranja, blanco, violeta y marrón); el resultado se informa por LCD y UART, y un servomotor se posiciona en el ángulo asociado al plástico detectado.

La iluminación se define en una tabla `COLORES[]` de cuatro colores de luz (Tabla V), que se aplican uno por vez sobre la muestra, manteniendo exactamente la misma iluminación durante la calibración y durante la medición. Cada plástico tiene así cuatro referencias y cuatro intervalos ADC, uno por cada iluminación. Una vez identificada la muestra, la iluminación del color detectado se mantiene fija como confirmación visual.

Cuadro V
COLORES DE ILUMINACIÓN Y ÁNGULO ASIGNADO

N.º	Color	R	G	B	Ángulo
1	Naranja	255	100	0	22°
2	Blanco	255	255	255	67°
3	Violeta	150	0	255	112°
4	Marrón	120	55	20	157°

Los ángulos se ordenan de derecha a izquierda del semicírculo. El servomotor es un SG90 posicional con escala semicircular de $180°$, por lo que el ángulo informado es el ordenado y no una posición medida. Al iniciar, el servomotor se posiciona en $90°$ (centro del recorrido).

Las principales conexiones utilizadas se presentan en la Tabla VI.

Cuadro VI
ASIGNACIÓN DE PINES DEL PROBLEMA C

Elemento	Arduino UNO	ATmega328P
LDR (divisor resistivo)	A0	PC0 / ADC0
LCD I2C (SDA)	A4	PC4 / SDA
LCD I2C (SCL)	A5	PC5 / SCL
Servomotor (señal)	D9	PB1 / OC1A
Tira WS2812 (DIN)	D6	PD6
LED RGB (R / G / B)*	D3 / D5 / D6	PD3 / PD5 / PD6
UART RX	D0	PD0 / RXD
UART TX	D1	PD1 / TXD

*Solo en el modo prueba (Sección de implementación).

La LDR se conecta formando un divisor resistivo VCC–$10\,k\Omega$–PC0–LDR–GND, y la LCD, conectada mediante un expansor PCF8574 (dirección 0x27), utiliza la librería provista por el docente.

VII-B. Estrategia de clasificación

Calibración. La calibración se realiza con las muestras reales colocadas en el montaje. Por cada plástico se ejecutan 16 barridos de las cuatro iluminaciones, promediando 16 lecturas ADC por color. De cada serie se extraen la referencia (promedio), el mínimo y el máximo, y el intervalo se amplía con un margen configurable ($MARGEN_ADC = 8$) para absorber pequeñas variaciones de posición. El sistema admite de uno a cuatro patrones; para esta práctica se calibran los cuatro, asociados a los ángulos del servomotor $22°$ (naranja), $67°$ (blanco), $112°$ (violeta) y $157°$ (marrón).

El código no contiene ningún valor ADC inventado o precargado: los valores absolutos dependen de las tolerancias de cada LDR, la intensidad de los LEDs, la distancia al sensor y la luz ambiente. Lo que identifica al plástico no es un número absoluto sino el patrón relativo de las cuatro lecturas, que solo puede obtenerse midiendo las muestras reales en el montaje definitivo. Por la misma razón, la calibración se invalida automáticamente si se modifica la tabla de colores, la cantidad de LEDs o el margen, mediante una firma de configuración con hash sobre `COLORES[]`.

Las calibraciones se guardan en EEPROM con número mágico, versión, firma de configuración y checksum, validando al cargar la coherencia de los rangos:

$$min \le ref \le max \le 1023 \tag{9}$$

De esta forma se calibra una sola vez y se opera en sesiones posteriores sin recalibrar. Durante la calibración física, el comando `p` informa la referencia y el rango de cada plástico para cada una de las cuatro iluminaciones.

Algoritmo. Al ordenar una medición, el sistema realiza un barrido de las cuatro luces y entrega el vector de lecturas L_i a una función pura de clasificación. Un plástico k se considera candidato únicamente si las cuatro lecturas caen simultáneamente dentro de sus intervalos:

$$min_{k,i} \le L_i \le max_{k,i} \qquad \forall\, i = 1, \ldots, 4 \tag{10}$$

El resultado puede ser:

- **Plástico identificado:** un solo candidato coincide con sus cuatro rangos. Se gira el servomotor al ángulo asignado ($22°$, $67°$, $112°$ o $157°$) y se fija la iluminación del color detectado.
- **Ambiguo (-2):** dos o más plásticos coinciden. Se reporta, se apaga la iluminación y no se actúa el servomotor.
- **Fuera de rango (-1):** ningún candidato coincide. Se apaga la iluminación y se conserva la posición del servomotor.

Se decidió no asignar el color más cercano ante una muestra desconocida: reportar «no identificado» es preferible a entregar una clasificación sin respaldo.

VII-C. Máquina de estados

La lógica del sistema se organiza mediante cuatro estados:

- **REPOSO:** el sistema espera comandos por UART y la LCD rota las páginas del último resultado. Los comandos se reciben por interrupción y se procesan solo en este estado; mientras el sistema está ocupado, las teclas nuevas se ignoran, evitando operaciones de calibración o medición superpuestas. La calibración (teclas `1`–`4`), los comandos de consulta y diagnóstico (`p`, `h`, `x`, `a`, `t`) y el comando `m` sin calibraciones cargadas se ejecutan sin abandonar este estado.

- **MIDIENDO:** se ilumina la muestra con cada uno de los cuatro colores, uno por vez, y se registra la lectura de la LDR. Es el único estado con el ADC activo.
- **CLASIFICANDO:** el vector de lecturas se compara con los intervalos calibrados de los plásticos almacenados en EEPROM.
- **MOSTRANDO:** se informa el resultado por LCD y UART, se posiciona el servomotor y se mantiene la iluminación del color detectado.

Al energizarse, y luego de inicializar los periféricos y cargar las calibraciones desde la EEPROM, el sistema parte de REPOSO. El comando `m`, con patrones válidos, lleva a MIDIENDO; al completarse el barrido se pasa a CLASIFICANDO, luego a MOSTRANDO y, una vez publicado el resultado, el sistema vuelve a REPOSO para procesar una nueva muestra. El diagrama indica además el estado de cada periférico en cada estado (1 = activo, 0 = inactivo).

Acceder a la imagen y al archivo Draw.io en GitHub

VII-D. Implementación y comunicación

El programa fue implementado en lenguaje C para el microcontrolador ATmega328P. El conversor ADC adquiere, por el canal ADC0, el valor de la LDR durante cada etapa de iluminación, con AVcc como referencia y prescaler 128. Cada lectura es el promedio de 16 muestras y, antes de leer, se espera un tiempo de estabilización de 180 ms tras cambiar la iluminación para que la LDR se estabilice.

El servomotor se controla con PWM por hardware mediante Timer1 en modo Fast PWM con $ICR1 = 4999$ (OC1A en PB1) y prescaler 64, lo que da un período de 20 ms con ticks de 4 μs. El ancho de pulso varía linealmente entre 1000 y 2000 μs para recorrer 0° a 180°, y se carga en `OCR1A`. Estos límites deben ajustarse al datasheet del servomotor utilizado.

La comunicación UART se configuró a 9600 baudios con formato 8N1 y recepción por interrupción. Los comandos solo se aceptan cuando el sistema no está ocupado y se procesan en el estado REPOSO.

Salida de iluminación. Todo el programa opera exclusivamente a través de las funciones `ws_show()` y `ws_apagar()`, lo que permitió intercambiar la salida física sin modificar el resto del código mediante la constante `MODO_RGB`:

- **Modo prueba** (`MODO_RGB 1`): LED RGB de cuatro patas (cátodo común) en D3/D5/D6, utilizado para desarrollar y verificar la lógica completa antes de contar con la tira WS2812. El LED RGB solo enciende o apaga cada canal, por lo que no reproduce el brillo exacto de la Tabla V.
- **Modo laboratorio** (`MODO_RGB 0`): tira WS2812 en PD6, manejada por una función en ensamblador GNU AVR integrada en `main.c`. Genera las tramas GRB a 800 kHz con temporización por ciclos de CPU (20 ciclos por bit) y con las interrupciones deshabilitadas durante el envío. La cantidad de LEDs se define con la constante `N_LEDS` (1 en la configuración utilizada). El código de la tira se selecciona cambiando un solo `#define` y recompilando.

Interfaz de usuario. Los comandos se envían por UART como una sola tecla (en mayúscula o minúscula, ignorando el Enter) y se resumen en la Tabla VII.

Cuadro VII
COMANDOS UART DEL PROBLEMA C

Comando	Acción
`1–4`	Calibra cada plástico
`m`	Mide y clasifica
`p`	Lista calibraciones y rangos
`x`	Borra las calibraciones
`a`	Lectura directa del ADC (diagnóstico)
`t`	Prueba de la iluminación
`h`	Muestra la ayuda

Tras cada medición, la LCD presenta cinco páginas rotativas: la primera con el color y el ángulo resultantes, y las otras cuatro, una por iluminación, con la lectura actual, la referencia y la diferencia. Si la muestra no es identificada, la referencia y la diferencia se muestran como no disponibles, y el diagnóstico se realiza comparando las lecturas con los rangos calibrados que lista el comando `p`.

VII-E. Montaje y resultados

Como criterio de validación del montaje óptico se estableció que las referencias calibradas caigan en la franja media del ADC, sin saturar en 1023 ni pegarse a 0, y que los rangos de plásticos distintos queden claramente separados, utilizando los comandos `a` y `p` como herramientas de diagnóstico.

La distancia entre la muestra y la LDR se absorbe en la calibración siempre que se mantenga fija entre la calibración y la medición, y la luz ambiente debe minimizarse (cámara oscura) para no desplazar las lecturas.

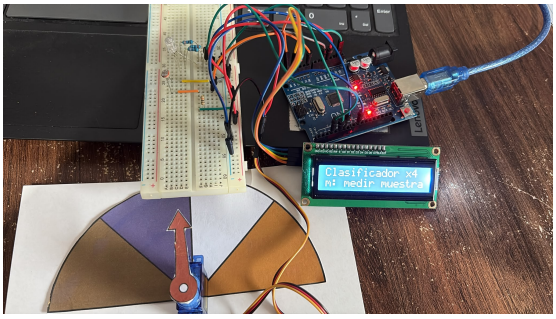


Figura 3. Montaje físico.

Durante las pruebas se comprobó la correcta iluminación de la muestra, la lectura de la LDR, la clasificación de los cuatro plásticos calibrados y el posicionamiento del servomotor en el ángulo correspondiente.

VII-F. Monitoreo de variables

Durante el funcionamiento, el sistema transmite por UART el color identificado, la lectura de la LDR para cada iluminación junto con su referencia, diferencia y rango (cuando la

muestra es identificada), y el ángulo ordenado al servomotor. La LCD presenta además el resultado y las páginas de diagnóstico para una verificación local del proceso de clasificación.

Ver captura de monitoreo

VII-G. Simulación

Antes de realizar la implementación física se verificó la lógica mediante simulación en Proteus y con el modo prueba de LED RGB, que permitió comprobar la secuencia de iluminación, la lógica de clasificación y el movimiento del servomotor. En la simulación se observó el arranque del sistema: el menú de ayuda en la terminal virtual, el mensaje inicial de la LCD («Clasificador x4» / «Calibrar 1 a 4») y el servomotor en su posición central de $90°$.

En Proteus el ADC es real y no se generan muestras ficticias. Como el modelo no vincula ópticamente el LED con la LDR, los niveles ADC se simulan manualmente modificando la iluminación de la LDR, por lo que la simulación valida la lógica pero no la detección óptica real.

Acceder al archivo de Proteus y capturas de la simulación

VII-H. Limitaciones

La precisión de la clasificación depende de la calibración con muestras reales y de que la distancia entre la muestra y la LDR se mantenga fija entre la calibración y la medición. Los valores ADC absolutos dependen de las tolerancias de cada LDR y de la intensidad de los LEDs, por lo que cualquier cambio en el montaje exige recalibrar.

La luz ambiente puede desplazar las lecturas respecto de las registradas en la calibración, por lo que se recomienda una cámara oscura. También pueden existir variaciones entre muestras de un mismo color debido al espesor, la transparencia y las características propias del material.

El ángulo mostrado es el ordenado al servomotor, no una posición medida. El sistema rechaza las muestras ambiguas o desconocidas en lugar de asignarles el color más cercano, por lo que una muestra fuera de los intervalos calibrados no es clasificada.

Además, cada operación de medición o calibración bloquea la atención de nuevos comandos hasta finalizar (unos segundos por barrido), y la calibración completa de un plástico requiere 16 barridos de las cuatro iluminaciones.

VII-I. Video de funcionamiento

Como evidencia final se incluye un video donde se observa la iluminación de las muestras, la adquisición de los valores mediante la LDR, la identificación del color, la información enviada por UART y LCD y el movimiento del servomotor.

Ver video en Google Drive

VII-J. Repositorio GitHub

El código fuente correspondiente al Problema C se encuentra disponible en el repositorio GitHub del laboratorio.

Acceder al código fuente

VIII. Problema D: Piano electrónico con ATmega328P

VIII-A. Descripción y funcionamiento

El sistema implementa un piano electrónico de ocho notas basado en el microcontrolador ATmega328P. Cada pulsador reproduce una nota de la escala de Do mayor, desde Do4 (262 Hz) hasta Do5 (523 Hz), y dos buzzers pasivos permiten una polifonía simple: el primero reproduce la melodía y el segundo el bajo o acompañamiento.

El sistema almacena además tres canciones predefinidas, que se seleccionan mediante comandos enviados por UART desde una computadora. Una pantalla LCD 16x2 muestra la nota pulsada con su frecuencia o, en modo canción, el nombre de la canción, la nota en curso y el avance de la reproducción.

Las canciones se eligieron entre obras de dominio público que no figuraban en la lista de canciones ya asignadas a otros grupos del curso (Tabla VIII).

Cuadro VIII
Canciones predefinidas del Problema D

Comando	Canción	Autor	Pasos
`C1`	Para Elisa	L. v. Beethoven (WoO 59)	123
`C2`	Arroz con leche	Tradicional	72
`C3`	Marcha nupcial	F. Mendelssohn (Op. 61)	68

Las conexiones utilizadas se presentan en la Tabla IX.

Cuadro IX
Asignación de pines del Problema D

Elemento	Arduino UNO	ATmega328P
Pulsadores Do, Re, Mi, Fa, Sol, La	D2–D7	PD2–PD7
Pulsador Si	D8	PB0
Pulsador Do agudo	D12	PB4
Buzzer 1 (melodía)	D9	PB1 / OC1A
Buzzer 2 (bajo)	D11	PB3 / OC2A
LCD RS	A4	PC4
LCD E	D10	PB2
LCD D4–D7	A0–A3	PC0–PC3
UART RX	D0	PD0 / RXD
UART TX	D1	PD1 / TXD

Los pulsadores se conectan entre cada pin y GND, utilizando las resistencias de *pull-up* internas del microcontrolador, por lo que no requieren componentes adicionales y se leen como activos en nivel bajo. Cada buzzer se acciona mediante un transistor 2N2222 en configuración de emisor común: el pin del microcontrolador se conecta a la base a través de una resistencia de $1\,k\Omega$, el emisor a GND y el buzzer entre $+5\,\mathrm{V}$ y el colector.

VIII-B. Estrategia de funcionamiento

La característica central del diseño es que cada buzzer dispone de su propio temporizador, configurado en modo CTC con conmutación automática de la salida (*toggle on compare match*). De este modo el periférico genera por hardware una onda cuadrada con ciclo de trabajo del 50 %, sin intervención

del procesador, y las dos voces pueden sonar simultáneamente. La frecuencia de salida es:

$$f = \frac{F_{CPU}}{2\,N\,(OCR+1)} \qquad \Rightarrow \qquad OCR = \frac{F_{CPU}}{2\,N\,f} - 1 \quad (11)$$

donde N es el prescaler. El Timer1 (16 bits) genera la melodía con $N = 8$; para Do4 se obtiene $OCR1A = 3816$, con una frecuencia real de 261,99 Hz y un error de 0,14 % respecto de la nota teórica (261,63 Hz). El Timer2 (8 bits) genera el bajo; como su registro admite como máximo 255, el programa selecciona para cada nota el menor prescaler cuyo valor de comparación entra en el registro. Para Do3 (130,81 Hz) resulta $N = 256$ y $OCR2A = 238$, con un error de 0,05 %.

La onda cuadrada simétrica responde al requisito de sonido claro de la consigna. Además, en modo canción, para que dos notas iguales consecutivas se distingan, cada voz que ataca una nota nueva en el paso siguiente se silencia 18 ms antes de finalizar el paso actual.

El sistema opera en dos modos:

- **Modo piano:** al presionar un pulsador, el buzzer 1 reproduce la nota y el buzzer 2 la misma nota una octava abajo. Si se presionan dos o más pulsadores a la vez, el buzzer 1 toma la nota más aguda y el buzzer 2 la más grave, formando un intervalo.
- **Modo canción:** se recorre una tabla almacenada en memoria flash, en la que cada paso indica la nota de la melodía, la nota del bajo y la duración. Un valor especial permite sostener una voz mientras la otra cambia de nota.

Los comandos UART admitidos se resumen en la Tabla X. No distinguen mayúsculas de minúsculas, ignoran los espacios y finalizan con un salto de línea.

Cuadro X
COMANDOS UART DEL PROBLEMA D

Comando	Acción
`C1`	Reproduce Para Elisa
`C2`	Reproduce Arroz con leche
`C3`	Reproduce Marcha nupcial
`S` o `STOP`	Detiene y vuelve al modo piano
`?` o `AYUDA`	Muestra la ayuda

VIII-C. Máquina de estados

La lógica del sistema se organiza en tres estados:

- **INIT:** corresponde a la secuencia de arranque. Configura los temporizadores, los pulsadores, la UART y la LCD, habilita las interrupciones y muestra un mensaje de bienvenida durante 1,5 s, tanto en la LCD como por UART junto con la lista de comandos. Luego pasa a modo piano.
- **PIANO:** lee los pulsadores y hace sonar la nota correspondiente, actualizando la LCD y la UART en cada cambio.
- **CANCIÓN:** reproduce la canción seleccionada con melodía y bajo. Durante la reproducción los pulsadores se ignoran.

Las transiciones son las siguientes: desde PIANO, los comandos `C1`, `C2` o `C3` llevan a CANCIÓN; desde CANCIÓN, otro comando de canción reinicia la reproducción con la nueva selección, mientras que el comando `S` o el fin de la canción devuelven el sistema a PIANO. Los comandos `S` en modo piano y `?` no modifican el estado. La canción en curso se guarda en una variable, por lo que un único estado CANCIÓN cubre las tres canciones de forma equivalente a tres estados separados.

La UART se atiende en los estados PIANO y CANCIÓN. Esto es posible porque la temporización de las notas no utiliza demoras bloqueantes: la duración de cada nota se mide comparando con una base de tiempo de 1 ms, y las únicas esperas son las cortas de la LCD (de microsegundos a pocos milisegundos). Por eso el comando `S` se procesa de inmediato aun en medio de una canción. Durante el mensaje de bienvenida los comandos quedan almacenados en el buffer de recepción y se procesan al terminar.

Acceder al diagrama de estados (captura y Draw.io)

VIII-D. Implementación y comunicación

El sistema fue implementado en lenguaje C para el microcontrolador ATmega328P y compilado en Microchip Studio. Los recursos utilizados son:

- **Timer0:** base de tiempo de 1 ms en modo CTC (prescaler 64, $OCR0A = 249$). Su interrupción incrementa un contador de milisegundos que utilizan el antirrebote y el reproductor de canciones.
- **Timer1:** generación de la melodía en OC1A (PB1), modo CTC con conmutación de la salida y prescaler 8.
- **Timer2:** generación del bajo en OC2A (PB3), modo CTC con prescaler variable según la nota.
- **USART0:** 9600 baudios, formato 8N1, con doble velocidad habilitada ($U2X0 = 1$, $UBRR0 = 207$, error de 0,16 %).

Los pines de los buzzers no son intercambiables: en modo CTC la conmutación automática se asocia a las salidas OC1A y OC2A, cuyo valor de comparación es OCR1A y OCR2A, lo que fija los buzzers en D9 y D11. El pin D13 no se utiliza para un pulsador porque en algunas placas el LED integrado puede alterar la lectura con *pull-up* interno.

El antirrebote se resuelve por software: los pulsadores se leen cada 5 ms y un cambio se acepta únicamente cuando, tras detectarlo, se repiten cuatro lecturas iguales, es decir, 20 ms de señal estable.

La recepción y la transmisión UART funcionan mediante interrupciones y buffers circulares de 32 y 128 bytes. A 9600 baudios cada carácter demora cerca de 1 ms, por lo que enviar un mensaje esperando la bandera de transmisión detendría el programa varios milisegundos y alteraría el ritmo de la canción; con el buffer, el programa solo copia el texto y la interrupción lo envía (salvo que el mensaje supere la capacidad

del buffer, como la ayuda, en cuyo caso espera a que se libere espacio). El sistema responde por UART a cada comando e informa cada nota tocada:

```
Tecla: Sol4 (392 Hz)
>Reproduciendo C2: Arroz con leche
```

La pantalla LCD se maneja en modo de 4 bits del controlador HD44780, con el pin RW conectado a GND. Como RS (PC4) y las líneas de datos (PC0–PC3) comparten el puerto C, los datos se escriben mediante lectura-modificación-escritura para no alterar RS durante el envío de cada nibble. La pantalla solo se reescribe cuando cambia la información, evitando parpadeos y demoras innecesarias.

Las tablas de Para Elisa y la Marcha nupcial se obtuvieron a partir de partituras de dominio público en formato MIDI del Mutopia Project, mediante un script en Python que separa la voz más aguda como melodía y la más grave como bajo y cuantiza los tiempos. Arroz con leche se transcribió a partir de una partitura de la Kodály Collection. Cada canción tiene su propia unidad de tiempo: 160 ms para Para Elisa (semicorchea), 210 ms para Arroz con leche (corchea) y 45 ms para la Marcha nupcial (doceavo de negra).

Durante el desarrollo se resolvieron además los siguientes problemas:

- **Polifonía:** generar las notas con demoras por software impedía hacer sonar dos voces y atender la UART a la vez; se resolvió generando cada voz por hardware con un temporizador independiente.
- **Resolución del Timer2:** con un prescaler fijo, las notas graves no entraban en el registro de 8 bits; se implementó la selección automática del prescaler.
- **Notas repetidas:** dos notas iguales seguidas sonaban como una sola nota larga; se agregó un silencio de articulación de 18 ms.
- **Cambio de entradas:** el diseño inicial utilizaba un teclado matricial; al pasar a ocho pulsadores independientes se reasignaron los pines, trasladando RS de la LCD a A4 para liberar D8.
- **Corriente de los buzzers:** los buzzers pasivos magnéticos presentan una resistencia de unos 16 Ω, por lo que conectados directamente superarían los 40 mA admitidos por un pin; el transistor 2N2222 entrega la corriente desde la fuente y el pin solo aporta aproximadamente 4 mA a la base.

VIII-E. Montaje y resultados

Antes del montaje físico, el programa compilado se verificó en un simulador del ATmega328P que ejecuta el mismo archivo `.hex`, complementado con la simulación en Proteus descrita más adelante. En la simulación se accionaron los pulsadores, se enviaron comandos por UART y se midió la frecuencia de la señal en los pines de ambos buzzers.

En modo piano, cada pulsador generó la frecuencia esperada (por ejemplo, 392,0 Hz para Sol4 y 523,0 Hz para Do5), con el bajo una octava abajo, y la pulsación simultánea de Do y Mi produjo el intervalo Mi4 + Do4. La LCD mostró en cada caso la nota y su frecuencia.

Las tres canciones se compararon nota por nota con sus tablas: Para Elisa (86 notas de melodía y 51 de bajo), Arroz con leche (72 y 28) y la Marcha nupcial (52 y 35) se reprodujeron sin diferencias de nota ni de duración, con duraciones totales de 20,6 s, 18,5 s y 27,5 s respectivamente. Al finalizar cada canción el sistema volvió al modo piano.

El comando `S` detuvo la reproducción de inmediato en medio de una canción, los pulsadores accionados durante una canción fueron ignorados y los comandos inválidos se respondieron con un mensaje de error sin alterar el estado del sistema.

El montaje físico se muestra en la Fig. **??**: los ocho pulsadores se conectan entre cada pin y GND, la LCD opera en modo de 4 bits y cada buzzer se acciona mediante su transistor 2N2222, según la Tabla IX.

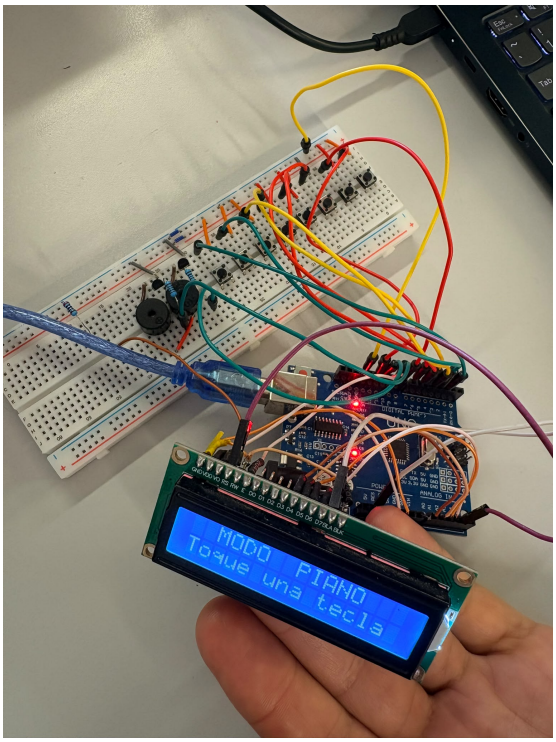


Figura 4. Montaje físico.

VIII-F. Monitoreo de funcionamiento

La comunicación UART permite seguir el funcionamiento desde la computadora: al iniciar se muestra la lista de comandos disponibles, cada pulsación informa la nota y su frecuencia, y cada comando recibe una confirmación (inicio de canción, canción detenida, fin de canción o comando desconocido).

En paralelo, la LCD presenta en la primera línea el modo o el nombre de la canción. En la segunda línea muestra, en modo piano, la nota pulsada con su frecuencia (o el intervalo, si hay dos pulsadores) y, en modo canción, la nota en curso junto con una barra de avance de ocho posiciones.

Ver captura de monitoreo

VIII-G. Simulación en Proteus

Para la simulación en Proteus se utilizan la placa Arduino UNO, ocho pulsadores, la pantalla LM016L, dos transistores 2N2222 con sus resistencias de base, dos buzzers y una terminal virtual a 9600 baudios conectada a los pines RX y TX. Esta simulación permite comprobar la lectura de los pulsadores, la generación de las frecuencias, la selección de canciones mediante UART y la información mostrada en la LCD. En la captura se observa el arranque del sistema: la terminal virtual con la lista de comandos y el mensaje «Modo PIANO», y la LCD en modo piano.

Acceder al archivo de Proteus y capturas de la simulación

VIII-H. Limitaciones

La onda cuadrada genera un timbre con abundantes armónicos, y los buzzers pasivos responden mejor en su frecuencia de resonancia, de algunos kilohertz, por lo que las notas graves del bajo pueden sonar con menor volumen que la melodía.

La polifonía está limitada a dos voces, ya que solo dos temporizadores pueden generar la onda por hardware mientras el tercero se reserva para la base de tiempo; en modo piano, con tres o más pulsadores simultáneos solo suenan la nota más aguda y la más grave. El Timer2, de 8 bits, ofrece además menor resolución en frecuencia que el Timer1.

Las canciones están almacenadas en memoria flash, por lo que agregar o modificar una canción requiere recompilar el programa. Por otra parte, las versiones de dos voces son reducciones de las partituras originales.

VIII-I. Video de funcionamiento

Como evidencia final se incluye un video demostrativo donde se observa la reproducción individual de las notas, la selección de las tres canciones mediante UART, la detención con `S`, el funcionamiento de los dos buzzers y la información presentada en la pantalla LCD.

Ver video funcionamiento

VIII-J. Repositorio GitHub

El código fuente correspondiente al Problema D se encuentra disponible en el repositorio GitHub del laboratorio.

Acceder al código fuente

IX. Gestión del código con GitHub

Los códigos del Laboratorio 3 se almacenan en el repositorio del curso para mantener trazabilidad de las modificaciones realizadas durante el desarrollo. El código de cada problema se organiza en su propia carpeta, junto con los diagramas de estados y las simulaciones correspondientes.

Acceso al repositorio GitHub del Laboratorio 3

Se registran modificaciones significativas mediante commits separados, de modo que pueda identificarse la evolución de cada problema y la intervención de los integrantes del equipo.

Para el Problema B, entre las modificaciones realizadas se encuentran la implementación inicial de ADC y control del motor, la incorporación del setpoint por UART, la introducción de límites de posición y zona muerta, el ajuste del PWM mínimo y la representación gráfica mediante MATLAB.

Para el Problema D, las tres modificaciones significativas registradas son: la implementación del modo piano con los pulsadores, la generación de notas mediante el Timer1 y la pantalla LCD; la incorporación de la comunicación UART con los comandos de selección y la primera canción; y el agregado de la segunda voz mediante el Timer2, las tres canciones definitivas y la máquina de estados completa.

X. Resultados generales

El Laboratorio 3 integra diferentes periféricos del ATmega328P dentro de aplicaciones de control. El Problema A combina adquisición de temperatura, salidas de control y comunicación; el Problema B integra ADC, UART, PWM y control de motor mediante puente H; el Problema C incorpora iluminación RGB, adquisición óptica y servomotor; y el Problema D utiliza pulsadores, temporizadores, buzzers, UART y LCD.

En el Problema A se verificó experimentalmente la respuesta ante variaciones escalón de temperatura, el cambio de estado en el muestreo siguiente a cada cruce de umbral, la modificación del punto medio por UART con rechazo de valores y cadenas inválidas, y el paso al estado de error ante la desconexión del sensor, con recuperación automática al restablecerla.

En el Problema B se verificó experimentalmente el funcionamiento del control de iluminación, incluyendo la modificación del setpoint, la lectura de posición, el cambio de sentido del motor, la zona muerta y los límites del mecanismo.

En el Problema C se comprobó en el montaje físico la iluminación de la muestra con las cuatro luces, la lectura de la LDR, la clasificación de los cuatro plásticos calibrados y el posicionamiento del servomotor en el ángulo asignado, mientras que la simulación en Proteus validó la lógica de calibración, clasificación y visualización, aunque no la detección óptica real.

En el Problema D se verificó mediante simulación del programa compilado que cada pulsador genera la frecuencia correspondiente, que las tres canciones se reproducen con melodía y bajo simultáneos sin diferencias respecto de sus tablas y que los comandos UART permiten seleccionar y detener la reproducción en cualquier momento.

XI. Conclusiones

El Laboratorio 3 permitió aplicar de forma integrada los principales periféricos del ATmega328P (ADC, PWM, UART, temporizadores, TWI y EEPROM) en cuatro sistemas de naturaleza distinta, programados en lenguaje C. En todos los casos la organización del funcionamiento mediante máquinas de estados facilitó el diseño, la verificación y la documentación: cada sistema pudo analizarse estado por estado, y las transiciones y las salidas quedaron definidas sin ambigüedad.

La validación combinó simulación en Proteus, montaje físico, monitoreo por UART y registro del código en GitHub, lo que dio trazabilidad al proceso de desarrollo.

En el Problema A, la máquina de estados de tipo Moore resultó adecuada para el control de temperatura: el comportamiento fue verificable estado por estado, el punto medio configurable por UART se validó contra los límites físicos del sensor y el estado de error desactiva las salidas ante un fallo del DHT11, lo que da un comportamiento seguro ante fallas. La lectura del DHT11 con interrupciones deshabilitadas y tiempos de espera acotados evitó que el sistema se bloqueara con el sensor desconectado. Como limitaciones, el ventilador se emuló con LEDs y no se incorporó histéresis, lo que es aceptable para un control de confort con un sensor de resolución de 1 °C.

En el Problema B, el control con realimentación de la LDR permitió regular la apertura del mecanismo mediante un puente H; la zona muerta de ± 40 unidades ADC evitó movimientos continuos alrededor del setpoint, el PWM mínimo de 200 aseguró el par necesario para iniciar el movimiento y los límites de posición ($POS_{MIN} = 100$ y $POS_{MAX} = 900$) protegieron al mecanismo en sus extremos. El análisis del cálculo del PWM mostró que, con ese mínimo, el control se comporta en la práctica como un esquema de encendido y apagado con zona muerta, por lo que una mejora natural sería ajustar la ganancia o incorporar un control proporcional más progresivo. El monitoreo con Data Visualizer, la gráfica en MATLAB y la simulación en Proteus confirmaron que las decisiones del controlador coincidían con las condiciones de iluminación y posición medidas.

En el Problema C, la clasificación basada en el patrón de las lecturas de la LDR bajo cuatro iluminaciones distintas, con intervalos obtenidos por calibración sobre muestras reales y almacenados en EEPROM, evitó depender de valores absolutos inventados y permitió rechazar las muestras desconocidas o ambiguas en lugar de forzar una clasificación. El uso de una capa única de salida de iluminación permitió desarrollar la lógica con un LED RGB y luego pasar a la tira WS2812 sin modificar el resto del programa, y el servomotor se controló con PWM por hardware del Timer1. La principal limitación es la sensibilidad a la distancia y a la luz ambiente, que obliga a recalibrar ante cualquier cambio del montaje y a trabajar en una cámara oscura.

En el Problema D, asignar un temporizador independiente a cada buzzer en modo CTC con conmutación por hardware permitió obtener dos voces simultáneas sin carga para el procesador, con errores de frecuencia inferiores al 0,2 %. La medición de tiempos sin demoras bloqueantes y la transmisión UART mediante buffers e interrupciones mantuvieron el ritmo de las canciones y permitieron atender los comandos en todo momento, mientras que el uso de transistores 2N2222 protegió a los pines del microcontrolador frente a la corriente de los buzzers. La polifonía quedó limitada a dos voces por la cantidad de temporizadores disponibles.

En conjunto, el laboratorio mostró que la elección del periférico adecuado (temporizadores con salida por hardware, ADC con promediado, comunicación por interrupciones) y la inclusión de protecciones (tiempos de espera, límites de posición, validación de entradas y estados seguros ante fallas) son decisivas para lograr sistemas confiables sobre un microcontrolador de recursos limitados.

Referencias

[1] Microchip Technology Inc., *ATmega328P 8-bit AVR Microcontroller Datasheet*.

[2] UTEC, Ingeniería Mecatrónica, *Laboratorio 3 – Tecnologías de Microprocesamiento*, Semestre IV, 2026.

[3] STMicroelectronics, *L298 Dual Full-Bridge Driver Datasheet*.

[4] Worldsemi, *WS2812 Intelligent Control LED Datasheet*.

[5] Hitachi, *HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver Datasheet*.

[6] The Mutopia Project, *Für Elise (WoO 59) y Wedding March (Op. 61)*, partituras de dominio público. [En línea]. Disponible: https://www.mutopiaproject.org

[7] Kodály Collection, Holy Names University, *Arroz con leche*. [En línea]. Disponible: https://kodalycollection.org/song.cfm?id=1100